

GLO-4030/GLO-7030 : Apprentissage par réseaux de neurones profonds

Travail Pratique 2

Version 1.0

Date de remise : 26 avril 2026, 23h59

Vous devez fournir un rapport en un seul document (format pdf) et les fichiers de code dans un seul fichier compressé (format zip) sur le site de cours (monPortail). Les Jupyter Notebooks sont acceptés. Vous pouvez utiliser les bibliothèques **Poutyne** et **deeplib**, si vous le souhaitez. Si le code est manquant, nous pourrions retirer jusqu'à 60% du total. De plus, vous devez discuter les résultats obtenus dans le rapport écrit. Veuillez noter qu'une présentation déficiente dans le rapport (manque de clarté, orthographe et grammaire, police de caractère illisible sur figure, etc) pourra entraîner une pénalité allant jusqu'à 10 % de la note. **Pour les étudiants en GLO-7030**, le rapport doit aussi obligatoirement être formaté avec LaTeX. Plusieurs sites web offrent des outils gratuits pour faire l'édition en collaboration, notamment Overleaf. Assurez-vous que vos documents ne soient pas publics.

Les questions suivantes ne sont que pour les étudiants de GLO-7030 :

1. Question 2.d)
2. Question 3

1 Ajustement, normalisation et généralisation

(GLO-4030 : 40%, GLO-7030 : 30%)

Dans cette question, vous allez effectuer une classification fine d'espèces d'oiseaux. Pour commencer, téléchargez les images du jeu de données CUB-200¹. Classez les images par ordre alphabétique des noms de fichiers et utilisez les 15 premières images de chaque classe comme jeu de test, les autres comme jeu d'entraînement. Vous pouvez utiliser la fonction fournie dans le fichier `question_1.py` pour cela.

Ensuite, réalisez les entraînements suivants avec le modèle ResNet18 de `torchvision`² en respectant les configurations indiquées ci-dessous.

- 1) l'initialisation aléatoire par défaut ;
- 2) le modèle pré-entraîné, mais en *gelant* («*freeze*») tous les paramètres de convolution ;
- 3) le modèle pré-entraîné, mais en *gelant* uniquement les paramètres dans "layer1" ; et
- 4) le modèle pré-entraîné, mais en laissant tous les paramètres (incluant les couches de convolution) se faire ajuster par rétro-propagation.

De plus, ajoutez le cas de figure suivant :

- 5) un réseau pré-entraîné DinoV2 `small` de Hugging Face³, en gelant toutes les couches. Seule la couche linéaire à la tête du réseau sera adaptée lors de cet entraînement. Un exemple pour charger le modèle DinoV2 `small` est fourni dans le fichier `question_1.py`.

Votre code doit clairement indiquer chaque cas de figure 1)-5).

Quelques consignes supplémentaires :

1. Pour les cas 2) et 3), *gelez* les paramètres de batch norm correspondant ainsi que les paramètres de "conv1" et de "bn". Pour le cas 5), *gelez* tous les paramètres du backbone ("model.dinov2").
2. Pour réussir à entraîner le réseau, vous aurez à adapter la couche pleinement connectée à la fin du réseau.
3. Pour charger les images, vous pouvez utiliser la classe `ImageFolder`⁴.
4. Vous devrez aussi vous assurer que toutes les images ont la même taille initiale. Pour ce faire, redimensionnez les images en 224×224 pixels⁵.
5. Pour la normalisation, vous pouvez utiliser la transformation `Normalize`⁶.

a) (5 pts pour GLO-7030 / 10 pts pour GLO-4030)

Normalisation ImageNet

Normalisez les données selon les coefficients calculés sur ImageNet, comme indiqué dans le tableau 1. Effectuez les entraînements 1) à 5) avec cette normalisation. Présentez les résultats

1. https://www.vision.caltech.edu/datasets/cub_200_2011/

2. <https://pytorch.org/vision/stable/models.html>

3. https://huggingface.co/docs/transformers/model_doc/dinov2#transformers.Dinov2ForImageClassification

4. <https://pytorch.org/vision/stable/datasets.html#torchvision.datasets.ImageFolder>

5. <https://pytorch.org/vision/stable/transforms.html#torchvision.transforms.Resize>

6. <https://pytorch.org/vision/stable/transforms.html#torchvision.transforms.Normalize>

sous forme de tableau, en rapportant les exactitudes (*accuracy*) en entraînement et en test pour chaque configuration.

TABLE 1 – Coefficients de normalisation pour l’entraînement sur ImageNet.

	R	G	B
Moyenne	0.485	0.456	0.406
Écart-type	0.229	0.224	0.225

b) (5 pts pour GLO-7030 / 10 pts pour GLO-4030)
Normalisation CUB-200

Répétez les expériences 1) à 5), mais cette fois en normalisant selon les statistiques calculées sur CUB-200. Présentez un tableau des coefficients calculés, similaire à celui de tableau 1. Rapportez également les exactitudes (*accuracy*) en entraînement et en test pour chaque configuration.

c) (10 pts) Comparaison

Discutez des différences de résultats selon la normalisation appliquée (ImageNet ou CUB-200). Analysez aussi l’impact du type de pré-entraînement (ImageNet vs. DINOv2) par rapport à une initialisation au hasard. Comment peut-on expliquer cette différence de performance ?

d) (10 pts) Erreur d’annotation et généralisation

Pour examiner l’effet du pré-entraînement sur la généralisation, refaites les expériences 1) à 5), mais cette fois en introduisant du bruit dans les annotations du jeu d’entraînement. Les erreurs d’annotation peuvent influencer la performance du modèle, et cette question permet d’observer l’impact du pré-entraînement sur la généralisation du modèle. Utilisez la classe `NoisyLabelDataset` fournie pour ajouter du bruit à 10% des annotations du jeu d’entraînement. Ne modifiez pas le jeu de test. Présentez un tableau des exactitudes (*accuracy*) en entraînement et en test pour chaque configuration.

Que remarquez-vous ? Quel est l’impact des erreurs d’annotation sur les performances de chaque configuration comparativement aux questions précédentes ? Quelles configurations performant mieux ? Comment peut-on expliquer cette différence de performance ?

2 Classification de terrains

(GLO-4030 : 40%, GLO-7030 : 30%)

Dans cette question, vous allez comparer la performance de différents modèles pour la classification de terrains boréaux à partir de données proprioceptives collectées par un robot autonome. Ces données proviennent de l'article "Proprioception Is All You Need : Terrain Classification for Boreal Forests"⁷. Ces données incluent :

- Les mesures de l'Unité de Mesure Inertielle (IMU), qui enregistre l'accélération et la vitesse angulaire du robot ;
- le courant consommé par chaque moteur ; et
- la vitesse des roues.

Les données nécessaires sont fournies dans le fichier ZIP `data/borealtc.zip`, et le script `borealtc.py` permet de les charger et de les pré-traiter. Le jeu de données BorealTC comprend 116 minutes d'enregistrements réalisés par un robot autonome (Husky A200), traversant des terrains tels que la neige, la glace et l'argile limoneuse. La figure 1, extraite de⁸, illustre la plateforme robotique et quelques types de terrains.

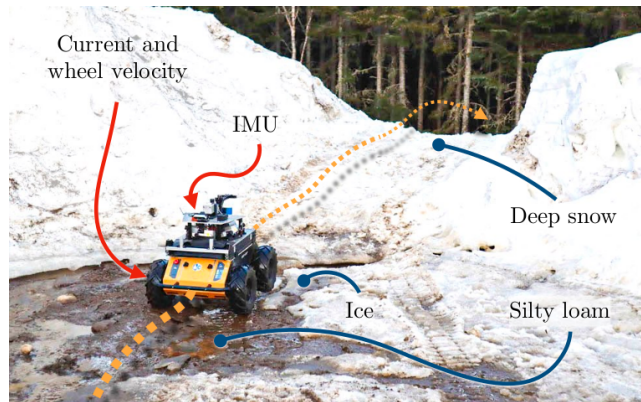


FIGURE 1 – Plateforme robotique collectant les données.

Le script `question_2.py` propose un exemple de chargement des données sous forme de séries temporelles avec le jeu de données BorealTC et la classe `SlidingWindowDataset`, qui segmente les données en fenêtres de 1.7s. Échantillonnées à une fréquence de 100 Hz, chaque fenêtre contient 170 mesures, chacune comportant 10 valeurs :

- Les vitesses angulaires en x , y et z ;
- les accélérations en x , y et z ;
- le courant consommé par les roues gauches et droites ; et
- les vitesses angulaires des roues gauches et droites.

Chaque donnée retournée par `SlidingWindowDataset` est un dictionnaire contenant

- `window` : un tenseur de taille 170×10 ;
- `class_name` : une chaîne de caractères représentant la classe (qui peut être convertie en entier avec `BorealTC.classe_to_idx`) ; et
- `run_id` : numéro de la séquence, n'est pas utile pour cette question.

7. <https://arxiv.org/abs/2403.16877>

8. <https://ieeexplore.ieee.org/abstract/document/10801407>

Pour toutes les questions, utilisez la fonction `split_train_test` fournie pour séparer l'ensemble d'entraînement de l'ensemble de test. Vous devrez trouver les paramètres de chacun de réseaux (e.g., nombre de couches, taille de dimensions cachées, nombre de têtes d'attention, etc.) vous-même. Remettez votre code dans le fichier `question_2.py`.

a) (5 pts pour GLO-7030 / 10 pts pour GLO-4030)
Approche RNN

Implémentez un réseau récurrent multicouche (RNN)⁹ capable de prédire le type de terrain pour chaque fenêtre de 1.7 s. Pour la classification, ajoutez une couche linéaire (*fully-connected*) prenant en entrée la dernière sortie du RNN (`output`, et non pas les états cachés).

Dans votre rapport, présentez :

1. Les paramètres utilisés pour le RNN ;
2. les courbes d'entraînement (perte et exactitude en fonction des époques, pour l'entraînement et la validation) ; et
3. l'exactitude finale sur l'ensemble de test.

b) (5 pts pour GLO-7030 / 10 pts pour GLO-4030)
Approche LSTM

Développez un réseau à mémoire longue et courte (LSTM)¹⁰ multicouche pour la même tâche de classification temporelle. Similairement au cas RNN, utilisez la dernière sortie du LSTM (`output`, et non pas les états cachés).

Dans votre rapport, présentez :

1. Les paramètres utilisés pour le LSTM ;
2. les courbes d'entraînement (perte et exactitude en fonction des époques, pour l'entraînement et la validation) ; et
3. l'exactitude finale sur l'ensemble de test.

c) (15 pts pour GLO-7030 / 20 pts pour GLO-4030)
Approche Transformer

Concevez un modèle basé sur un `TransformerEncoder`¹¹ pour prédire le type de terrain à partir des fenêtres temporelles. Pour ce faire, voici quelques consignes :

1. Chaque mesure (qui possède 10 valeurs) doit être projeté à l'aide d'une couche linéaire (*fully-connected*) pour produire un jeton (*token*) de taille `d_model`.
2. Ajoutez de l'encodage de position 1D avec `PositionalEncoding1D`¹².
3. Ajoutez un jeton de classification (`CLS`) au début de la séquence. Ce jeton devra être un `Parameter`¹³ appris par le réseau. Vous devrez répéter ce jeton pour chaque série

9. <https://pytorch.org/docs/stable/generated/torch.nn.RNN.html>

10. <https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>

11. <https://pytorch.org/docs/stable/generated/torch.nn.TransformerEncoder.html>

12. <https://github.com/tatp22/multidim-positional-encoding>

13. <https://pytorch.org/docs/stable/generated/torch.nn.parameter.Parameter.html>

temporelle dans votre mini-lot (*mini-batch*) avant de le concaténer à votre mini-lot.

4. Utilisez la sortie associée à ce jeton pour prédire la classe avec une couche linéaire pour chaque exemple des mini-lots.

Dans votre rapport, présentez :

1. Les paramètres utilisés pour le transformer ;
2. les courbes d'entraînement (perte et exactitude en fonction des époques, pour l'entraînement et la validation) ; et
3. l'exactitude finale sur l'ensemble de test.

d) (5 pts GLO-7030 seulement) Comparaison

Comparez les trois approches (RNN, LSTM, Transformer) en termes de :

1. Performance (exactitude sur l'ensemble de test). Résumez vos résultats dans un tableau. Pourquoi obtenez-vous ces résultats ?
2. Convergence (rapidité de la diminution de la perte selon l'époque).
3. Complexité (nombre de paramètres) et le temps d'entraînement.

3 Où suis-je ? Classification d'images de ville par réseau de neurones (GLO-7030 : 40%)

Dans cette question, vous êtes confrontés à un problème de vision, dont le but est d'identifier une ville à partir d'une seule image. Pour ce faire, nous vous fournissons une base de données d'entraînement comportant environ 22 500 images uniformément réparties sur 5 classes, chacune correspondant à l'une des villes suivantes : Québec, Montréal, Boston, Paris et Londres. La Figure 2 montre des exemples d'images pour chaque ville.



FIGURE 2 – Exemples d'images dans la base de données.

Utilisez les techniques vues en classe pour entraîner le meilleur réseau possible pour la classification des images. Afin de pouvoir vous comparez à vos collègues en toute équité, le jeu de données de test n'est pas étiqueté. Vous obtiendrez votre score de test en soumettant vos prédictions sur un Leaderboard en ligne sur Kaggle, *pour un maximum de deux soumissions par jour par équipe*.

Tous les détails concernant les données d'entraînement, les données de test, le Leaderboard, le code de chargement de données, la génération du fichier de soumissions, etc, se trouvent sur le lien de la compétition : <https://www.kaggle.com/competitions/ou-suis-je-h-2026>.

Vous devez d'abord vous inscrire à la compétition en utilisant le lien privé suivant : <https://www.kaggle.com/t/d53a1b6ff0824ca794ff7b2a9bcb1987>. Ensuite formez votre équipe avant de soumettre (si vous êtes deux dans l'équipe du TP2, vous devez alors être dans la même équipe Kaggle, nous n'accepterons pas les fusions des équipes après votre première soumission). Des points bonus (15%, 10% et 5%) seront attribués aux trois premières équipes ayant les meilleurs résultats sur le Leaderboard final. Votre rapport doit aborder les éléments suivants :

- L'approche que vous avez suivie pour choisir l'architecture et la procédure d'entraînement.
- Le choix de vos hyperparamètres et comment vous les avez fixés.
- Les défis que vous avez rencontrés et comment vous les avez résolus.
- Les différentes itérations du développement de votre solution, i.e. les ajouts qui ont permis d'augmenter les performances (data augmentations, pré-entraînement, etc).
- Discussion sur les performances de votre réseau en entraînement et en validation. Notamment, produisez les matrices de confusion et commentez les patterns que vous y voyez.
- N'oubliez pas de mentionner le nom d'équipe que vous avez utilisé pour la compétition Kaggle!